TECHNICAL RESEARCH | API IMPLEMENTATION CHECK

QGIS Integration & FortyGuard API Check

Research findings and implementation approach
Prepared for Mustafa Hamdy | 12 October 2026

Result: The documented API and QGIS support a plausible direct integration. The official API documentation and public client agree on the request structure and polling pattern. Live API validation and an actual QGIS rendering test are still required before the integration can be called working. [1-4]

Scope and method

This research reviews QGIS integration, the documented FortyGuard API contract, the proposed implementation, and the technical questions that still require validation.

Reviewed the official API documentation supplied by Mustafa, QGIS documentation, the public Python client, the heatmap notebook, and API plan information. This was source research; no authenticated API request or QGIS runtime test was performed.

Main findings

Area	Finding	Status
Plugin route	QGIS supports Python plugins. A small PyQGIS interface can select an area and load results. [4]	Documented
API request	The client uses an api-key header and POST /v1/heatmap. It sends a GeoJSON area and nested date_time options. [2]	Source inspected
Asynchronous jobs	Submission returns a job ID; the client polls /v1/status/{activity_id}. [2]	Source inspected
Layer format	The heatmap notebook describes map_data as a GeoJSON FeatureCollection. QGIS can load that vector format. [3, 5]	Format supported
Deployment compatibility	Start on QGIS 3.44 LTR. Test QGIS 4.2 separately because QGIS 4 uses Qt6. [9, 10]	Runtime test pending

Temperature unit and current documentation

The current official Heatmap documentation explicitly defines tcm output in Celsius. This agrees with the notebook and README. A client.py docstring says Fahrenheit and appears inconsistent with the current documentation. Use Celsius as the documented contract; verify one real result before release and report the stale comment to the API team. [1-3, 14]

Also confirm plan naming and capabilities: the notebook uses Basic/Premium while the public pricing page uses Basic/Pro. Do not assume that a name or advertised resolution maps directly to the chosen test key. [3, 11]

API CONTRACT

Request structure found in the official client

The official Heatmap and Check Status pages document these operations, also found in the public client. This is documentation review, not a verified production response. [2, 14, 15]

Item	Observed value / path
Base URL	https://api.fortyguard.com
Create a heatmap	POST /v1/heatmap
Authentication	HTTP header api-key; JSON content type
Submission job ID	Raw response: data.activity_id
Job status	GET /v1/status/{activity_id}; raw response: data.status
Completed map	Client expects result under data; notebook reads result.map_data after the client removes the envelope.

Payload fields

Field	Purpose / expected shape
polygon_aoi	GeoJSON FeatureCollection containing the selected polygon; coordinates ordered longitude, latitude.
date_time.start_date	YYYY-MM-DD; current docs support 2019 onward and forecasts up to 12 hours ahead. [14]
date_time.filter_type	1: one hour; 2: hour range; 3: one day; 4: day range.
date_time.start_time date_time.end_time	Time fields needed for hour-based requests.
date_time.end_date	End date for a multi-day request.
granularity	The client/notebook describe 60, 80, or 100 meters; confirm current plan options.
analytic_type	Use tcm for the first temperature prototype. The current docs specify Celsius for tcm; the other analytical types measure hours. [14]

Polling details worth preserving

The client accepts completed/succeeded and failed/error, ignoring case. It polls every three seconds by default. An initial status 404 is retried within a time limit. Confirm these client behaviors against the service. [2]

Current docs list 3,500 credits per Heatmap request for Basic/Startup and 4,220 for Premium. Area caps are 10 and 50 square miles. Confirm the test key's credits before submitting. [14]

Do not repeat the heatmap POST just because polling fails. Retry status checks for the same job ID. Stopping local polling is not proof that the server job was cancelled; no server cancellation operation was confirmed in this review.

OUTPUT AND QGIS IMPLEMENTATION

Turn the result into a useful map layer

Select the correct output field

Request type	Notebook field mapping	Plugin action
Single hour / hour range	map_data.features[].properties.temperature	Use that field only if it exists and is numeric.
Day / day range	average_temperature, min_temperature, max_temperature	Choose one explicitly and include it in the layer name/legend.
Other analysis types	properties.value with stats_data.units	Exclude from the first prototype; do not color these as degrees.

The notebook also describes stats_data for area-wide statistics. Field names and units must be checked on an actual result before the plugin fixes a schema. The reviewed notebook has no stored execution outputs, so it did not provide a captured live response. [3]

Recommended component design

Component	Implementation responsibility
Small plugin interface	API key, one polygon selection, supported date/time options, Generate button, and status message.
Area preparation	Read the selected geometry on the main thread, validate it, transform to EPSG:4326, and serialize a GeoJSON area. [6]
API adapter	Use QgsNetworkAccessManager for requests, with asynchronous polling and a maximum waiting time. Repository guidance recommends the native network manager. [7]
Result handling	Inspect result fields, save the GeoJSON in a stable local location, and load it through QgsVectorLayer using the OGR provider. [5]
Styling	Apply a graduated color renderer to the chosen numeric field; show the confirmed units, date, and resolution. [12]
Credentials and errors	Start with a key held in memory. Consider QGIS authentication storage for later persistence; keep keys out of logs and saved projects. [8]

Network waits must not block the interface. QGIS layers, project objects, and widgets must be read/updated on the main thread. If a worker task is used, copy its inputs first and return plain results to the main thread. [13]

Minimum plugin package

metadata.txt describes the plugin and supported versions; __init__.py exposes classFactory; a main module manages the interface and lifecycle. Keep API calls, geometry handling, and result styling in separate modules. This structure follows QGIS plugin conventions; it is a proposed design, not implemented code. [4]

Use qgis.PyQt imports rather than assuming PyQt5. A QGIS 4 compatibility declaration must follow real testing, not just a metadata change. [9]

TECHNICAL ASSESSMENT

Findings, requirements, and validation gaps

Assessment: The documented GeoJSON output provides a suitable format for a QGIS vector layer. A direct Python plugin is a reasonable integration approach. A live API request and QGIS rendering test are still needed to confirm the full flow.

What the research establishes

Topic	Evidence and conclusion
Authentication and request	Official docs and public client agree on the api-key header, POST /v1/heatmap, polygon_aoi, and date_time. [2, 14]
Job processing	The API documents a status endpoint. The client shows a polling pattern that can be adapted to QGIS. [2, 15]
Map output	The docs describe map_data as GeoJSON polygons. QGIS supports loading a vector layer through OGR. [5, 14]
Temperature unit	Current Heatmap docs define tcm in Celsius. A conflicting client comment should be corrected and a real result checked. [2, 14]
Plugin structure	PyQGIS supports the proposed interface, geometry preparation, result loading, and styling. [4-8, 12]

Questions for the API owner

- Which regions and date ranges are enabled for the intended API key?
- What time zone should the plugin use for start_time and end_time?
- Which area cap, resolutions, credits, and request limits apply to that key?
- Can the team provide a current completed tcm response with its GeoJSON properties?
- Are MultiPolygon areas or multiple selected features supported, or should the first version accept one Polygon only?

How to validate the proposed integration

Use an authorized test key to submit one small tcm request. Save its activity_id, poll the status endpoint, and save the completed result. Check map_data geometry, numeric properties, and units. Load the GeoJSON in QGIS and verify its location, styling, and saved-project behavior.

Use a supported region confirmed by the team. The public quickstart uses U.S. examples; that is not sufficient evidence for worldwide coverage on every key. [1]

Research limitation

No authenticated API call or QGIS runtime test was performed. The findings support an implementation design; they do not establish a working plugin. The notebook has no stored execution outputs, so its field mapping still needs comparison with a real response.

REFERENCES

Official documentation and inspected source

Reviewed 12 October 2026. Source-code findings below came from downloaded public files, not from a live API session. Links are clickable.

[1] FortyGuard Temperature API quickstart - README

[2] FortyGuard Python client - authentication, payload, polling

[3] FortyGuard heatmap notebook - request options and result fields

[4] QGIS - Structuring Python plugins

[5] QGIS - Loading vector layers

[6] QGIS - Coordinate reference systems and transforms

[7] QGIS - Plugin guidelines and native network manager

[8] QGIS - Authentication infrastructure

[9] QGIS - QGIS 4 and Qt6 migration

[10] QGIS - Current downloads and LTR information

[11] FortyGuard - API plan and output guidance

[12] QGIS - Vector layers and graduated rendering

[13] QGIS - Background task and main-thread requirements

[14] FortyGuard API - Heatmap contract, units, dates, and credits

[15] FortyGuard API - Check Status

[16] FortyGuard API - Introduction supplied by Mustafa

Source-inspection record

The API site's public documentation asset was read as text. The downloaded client and notebook parsed successfully as Python and JSON. These checks validate readable source material, not API execution or QGIS compatibility.

Research outcome statement

Define a technically supported route for a FortyGuard QGIS heatmap connector, document the API request and result mapping from official source, identify a stale unit comment and current API limits, and specify the live checks needed before prototype delivery.

Technical decision: Proceed to the small implementation check. No required backend rewrite was established by this research, but live compatibility is still unconfirmed. Use the documented Celsius unit and verify it with a real result before release.